Document 15 — Five Projects (reference index)

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Not new content. This is a standing page you come back to — not something you read once and move past. Each project has its own full plan; this page just maps them out and shows the "plan before you code" steps (§17 of the original plan: Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks) you should redo fresh at the start of every project.

How to Read & Practice This Document

- **What:** the fixed process every project starts with, before any code: **Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks.**
- **Why:** at the start of every project, it's tempting to just start typing. Projects that skip straight to code end up as working demos with an architecture you can't explain in an interview.
- **When:** at the start of every one of the projects, every time, no skipping — including the capstone.
- **How to practice:** before opening your editor for a new project, write out the 7 steps above yourself, in your own words, for that specific project. Only then go to that project's folder and start.

Jump to: Project Index · Practice Exercise · Go Deeper

The Story — what this document is actually building

You've just finished (or are about to start) a run of real projects — five core ones, plus eight bonus ones — and it's easy to lose the thread of how they connect — each one has its own folder, its own README, its own requirements. This page is the map that ties them together: one table showing every project side by side, how many agents each one uses, which documents taught you what you needed for it, and what it's meant to prove you can do.

Notice the pattern in the "Agents" column: 2 → 2 → 3 → 5 → 5. Each project is deliberately not a huge jump from the last one — it adds *one* new idea (a second agent, an approval step, a supervisor) instead of several at once, so by the time you reach Project 4's five-agent team, none of the individual pieces are new to you, only the way they're combined.

The other half of this page is the 7-step process listed above — **Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks.** That process doesn't live in any one project's folder because it isn't specific to any one project — it's the habit you're meant to run, fresh, at the start of every single one, including the final capstone. This page is where you come back to re-read those 7 steps each time, not something you read once and file away.

Project Index

Every project is a real multi-agent system — the number of agents grows across the arc (2→2→3→5→5) so each project adds just one new idea instead of several at once. Full details: CURRICULUM.md §2.

#	Project	Agents	Folder	Built during	Proves
1	SupportDesk: Concierge & Triage	2	project_1_supportdesk_chat_and_triage	Docs 01-04	You can talk to a model correctly and safely, and route between two simple agents
2	ResearchHand: Worker & Verifier	2	project_2_researchhand_tool_agent	Docs 05-07	You can give a model real abilities, check its choices, and add an independent check
3	DocuMind: Retriever, Reasoner & Approval	3	project_3_documind_rag_agent	Docs 08-10	You can build agents as a clear, checkable graph, with real handoffs
4	ContentForge: Supervisor-Led Team	5	project_4_contentforge_multi_agent	Doc 11	You can split up a problem across specialist agents, with a supervisor deciding routing on the fly
5	ContentForge Pro: Production Platform	5 (production-ready)	project_5_contentforge_pro_production	Docs 12-19	You can ship, test, secure, watch, version, and defend the design of #4
6 (bonus)	LangChainPro: Production LCEL Patterns	—	project_6_langchainpro_lcel_patterns	Doc05	You can build a real multi-step pipeline (branching, parallel calls, retries/fallbacks) in LangChain alone, without needing LangGraph
7 (bonus)	MCPForge: MCP Server	— (server, not agents)	project_7_mcpforge_mcp_server	Doc06's MCP topics	You can expose real tools/resources/prompts over the Model Context Protocol, usable by any MCP client
8 (bonus)	MCPBridge: MCP Client Agent	1 (dynamic tools)	project_8_mcpbridge_mcp_client	Doc06/07's MCP + ReAct topics	You can build an agent whose tools are discovered at runtime from live MCP servers, not hardcoded
9 (bonus)	PromptShield: Injection-Hardened RAG Agent	1	project_9_promptshield_injection_defense	Doc06/08's prompt injection topics	You can defend a RAG agent against injected instructions, and prove it with a real red-team test suite
10 (bonus)	MemoryKeeper: Persistent Cross-Session Memory	1	project_10_memorykeeper_persistent_memory	Doc09's long-term memory topic	You can give an agent real memory that survives across separate conversations,

#	Project	Agents	Folder	Built during	Proves
					with a working "forget me"
11 (bonus)	MCPCrew: Multi-Agent Research Team	3 (+ Supervisor)	project_11_mcpcrew_multi_agent_mcp	Doc06/11's MCP + multi-agent topics	You can build a Supervisor-led team where each specialist connects to its own MCP server, with graceful degradation
12 (bonus)	MCPResearch: Agentic Pipeline as an MCP Tool	2 (Researcher + Fact-Checker)	project_12_mcpresearch_agentic_mcp_tool	Doc06/11's MCP + multi-agent topics	You can wrap a whole agentic pipeline behind one MCP tool call — the "agent-as-a-service" pattern
13 (bonus)	CodeGuard: PR-Review Team	4	project_13_codeguard_pr_review	All of Docs 01-19, used again	You can apply the whole architecture to a new problem, with less help from this curriculum

Practice Exercises

Where your answer lives: `15_five_projects_index/practice/` (`mkdir -p practice`). This exercise is written, not coded, so it's a Markdown file:

```
practice/
└── seven_step_practice.md   the 7 steps for the email scenario
```

Why this file exists:

- `seven_step_practice.md` — your own first run of the 7 steps on a real, underspecified request, written down so you can compare it with the worked example — and re-read it before starting each project.

Why only 1 here: this page's whole job is the 7-step process, not code — one worked example is enough to show you how to actually use it. Real practice is running these 7 steps yourself, fresh, at the start of every project.

Basic — run the 7-step process on a mini scenario

- **What:** given a one-sentence feature request, write out all 7 steps (Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks) yourself, on paper, before looking at a worked example.
- **Why:** reading the 7-step list isn't the same skill as actually using it under a real, underspecified request — the first time you try it for real shouldn't be at the start of Project 1.
- **When you'll hit this for real:** literally the first thing you do at the start of every project in this curriculum, and every new feature request for the rest of your career.
- **How to do it:** use this scenario — *"Build something that reads a team's daily emails and flags the urgent ones."* Write your own 7 steps first. Then compare against the worked example.
- **Save as:** `practice/seven_step_practice.md`.

- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Go Deeper (Optional)

No new reading — read Anthropic — Building Effective Agents again before Project 4, and once more before the capstone. It reads differently each time you've built more.

Move On When

This is a reference, not a checkpoint — there's nothing to "pass" here. Move on once you understand the 7-step process well enough to use it on your own, at the start of Project 1.

Each project's own folder has its full requirements, test cases, and hints. This index just tells you where things are, and what order they build in.